



Exploring Machine Learning Frameworks: PyTorch

Practical Notes

Introduction

In this exercise we will aim to implement a specific DNN in PyTorch. The practical is designed to give you experience with a common ML framework, and lets you experiment with, and compare performance for, different configurations/options for a simple ML task.

We will focus on implementing the DNN to recognise/categorise a set of images, using the EIDF system for developing and running the application. If you do not already have access to EIDF, or do not know how to get on to the system, please **let a helper or lecturer know** and they will guide you through the process.

Getting started

1. Log on to the EIDF VM using your student account
2. Copy the file `frameworks-pytorch-eidf.tar` from `/home/eidf018/eidf018/shared/mlas`
3. Unpack it with the command `tar xvf frameworks-pytorch-eidf.tar`

If you do not know how to do any of the steps above, or are struggling to complete them, please let a demonstrator know so they can help.

For the exercises we will use the both the CPUs in the login VM for this course (`eidf018-mlas`) and the GPU nodes on the EIDF. Example scripts to access the GPU nodes have been provided, but you may need to update the `yaml` script

to include your details. We will provide guidance for doing this later in this document.

Example DNN

For this practical exercise we will use the popular MNIST handwriting tasks as the target for the DNNs we will construct. It is a useful test for exploring DNN creation and use as it has relatively small datasets, and does not require a large DNN to obtain good results.

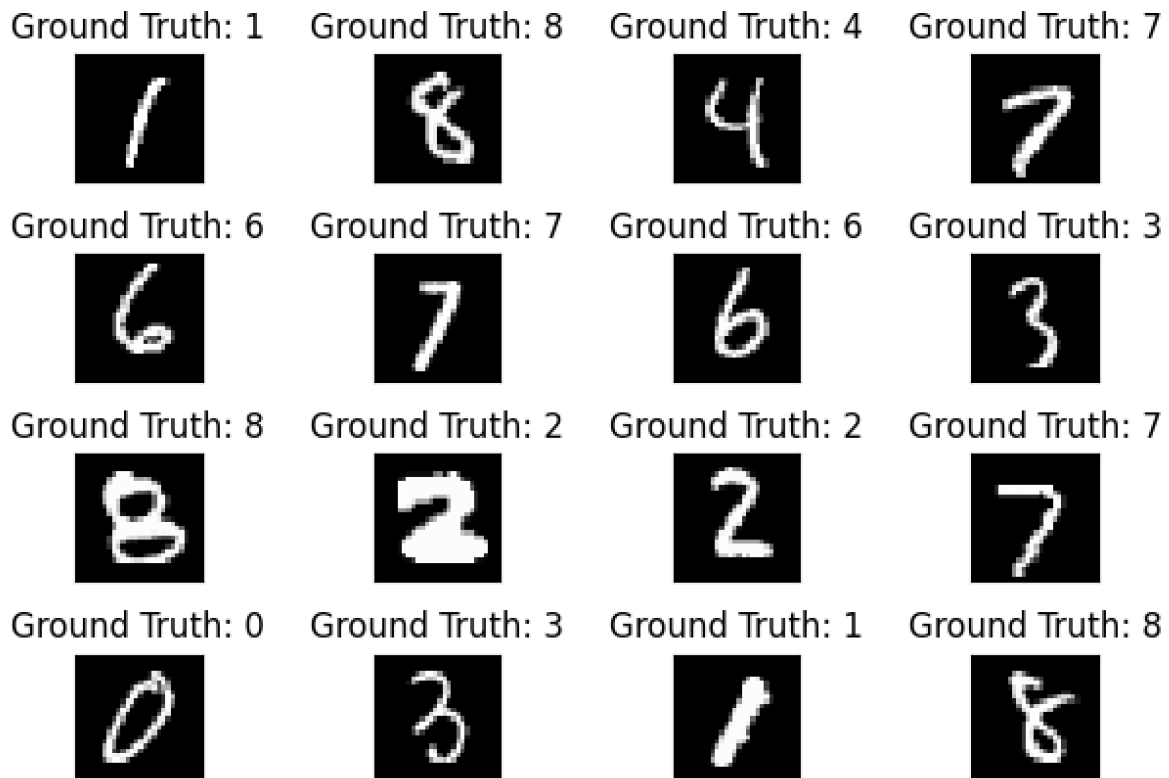
The test dataset consists of a number of images of handwritten numbers, with associated ground truth labels that can be used by the DNN for training and evaluating outputs.

This test case has been well benchmarked and explored for DNN and other ML approaches. Wikipedia (https://en.wikipedia.org/wiki/MNIST_database) lists the following as the classification accuracy achieved with a range of different methods:

Type	Classifier	Distortion	Preprocessing	Error rate (%)
Linear classifier	Pairwise linear classifier	None	Deskewing	7.6 ^[10]
K-Nearest Neighbors	K-NN with rigid transformations	None	None	0.96 ^[28]
K-Nearest Neighbors	K-NN with non-linear deformation (P2DHMDM)	None	Shiftable edges	0.52 ^[29]
Boosted Stumps	Product of stumps on Haar features	None	Haar features	0.87 ^[30]
Non-linear classifier	40 PCA + quadratic classifier	None	None	3.3 ^[10]
Random Forest	Fast Unified Random Forests for Survival, Regression, and Classification (RF-SRC) ^[31]	None	Simple statistical pixel importance	2.8 ^[32]
Support-vector machine (SVM)	Virtual SVM, deg-9 poly, 2-pixel jittered	None	Deskewing	0.56 ^[33]
Neural network	2-layer 784-800-10	None	None	1.6 ^[34]
Neural network	2-layer 784-800-10	Elastic distortions	None	0.7 ^[34]
Deep neural network (DNN)	6-layer 784-2500-2000-1500-1000-500-10	Elastic distortions	None	0.35 ^[35]
Convolutional neural network (CNN)	6-layer 784-40-80-500-1000-2000-10	None	Expansion of the training data	0.31 ^[36]
Convolutional neural network	6-layer 784-50-100-500-1000-10-10	None	Expansion of the training data	0.27 ^[37]
Convolutional neural network (CNN)	13-layer 64-128(5x)-256(3x)-512-2048-256-256-10	None	None	0.25 ^[22]
Convolutional neural network	Committee of 35 CNNs, 1-20-P-40-P-150-10	Elastic distortions	Width normalizations	0.23 ^[17]
Convolutional neural network	Committee of 5 CNNs, 6-layer 784-50-100-500-1000-10-10	None	Expansion of the training data	0.21 ^{[24][25]}
Random Multimodel Deep Learning (RMDL)	10 NN-10 RNN - 10 CNN	None	None	0.18 ^[27]
Convolutional neural network	Committee of 20 CNNs with Squeeze-and-Excitation Networks ^[38]	None	Data augmentation	0.17 ^[39]
Convolutional neural network	Ensemble of 3 CNNs with varying kernel sizes	None	Data augmentation consisting of rotation and translation	0.09 ^[40]

We will aim to create a DNN implementation of a network to classify input images into numbers. The input numbers are 28x28 pixel images, and their actual values are just represented as a single digit (0-9) for the corresponding image. The following figure show examples of the dataset:

First 16 images with their actual values



You will be provided with a directory containing 4 separate datasets:

- `x_train`: Source images for training
- `y_train`: Ground truth data for the training files
- `x_test`: Source images for testing
- `y_test`: Ground truth data for the testing files

The dataset is stored in `/home/eidf018/eidf018/shared/mlas/mnist` on the EIDF, and the example code you will use has been configured to automatically use this location. You will see from these directories that the training dataset has around 60,000 images, and the testing dataset has around 10,000 images.

For this exercise we will **create a DNN** that can take the images in the training dataset and then identify the numbers from those images. We can then validate our network using the test dataset.

We have provided initial implementations that already perform data loading and visualisation functionality. For the development of your networks it is sensible to only use a subset of the training and validation (`_train` and `_test`) datasets to

reduce the runtime. This can be done by passing the number of images to use as a command line argument, i.e.:

```
python3 pytorch-mnist.py 100
```

We also have provided functionality to visualise the first 16 images in the training dataset, along with their intended values. To enable this you need to pass `True` as the second argument to the program, i.e.:

```
python3 pytorch-mnist.py 100 True
```

For running on the EIDF GPUs using Kubernetes you will need to disable this functionality, this can be done by simply removing this argument from the command line.

Exercise 1: PyTorch

As we are using PyTorch for this exercise you will need to create a PyTorch virtual environment to be able to run the code. To do this, run the following commands on the EIDF `eidf018-mlas` VM:

```
python3 -m venv pytorch
source pytorch/bin/activate
pip install torch torchvision matplotlib
```

We have provided you an initial file with the data loading and visualisation functionality already enabled. You can test this by running the following command:

```
python3 pytorch-mnist.py 20 True
```

This may take a minute or two to complete, and it may also fail because of issues visualising the data (i.e. problems with display or X connections). If that happens let a lecturer/demonstrator know and we can help sort out any issues you have.

Inside the code you will see the places required to implement the DNN functions. At the moment the code only reads in the data and visualises it if the correct parameters are passed to the code. You will need to **implement the network**, **create an optimiser**, **define the training loop**, and also **define the evaluation loop** as well (which will be used to calculate how accurate the trained network is on predicting results for the test data set).

We are aiming to create a network that can take in a single dimension tensor of size 784 (28×28), and output 10 values representing the probability the input is 0-9. The output with the highest probability will be the prediction of the network.

Implementing the network

In the code there is already a class defined for the network functionality, called `Network`. In that you will see a basic 2 layer network has been created. This simply connects the input layer to the output layer via some intermediate connections.

Your first task is to add some more network layers. You do not need to add mean for now, just add a single network layer between the input and output. You can use the following code:

```
nn.Linear(hidden_sizes[0], hidden_sizes[1])
```

and update the last layer so it's first input parameter is `hidden_sizes[1]` rather than `hidden_sizes[0]`.

Next add a SoftMax activation function after the output layer, using this code:

```
nn.LogSoftmax()
```

This activation function will help with the categorisation output. Note, it is generally required to add activation functions after linear layers in the network. Try experimenting with the `nn.ReLU()` activation function after your other linear layers (not the one with the Soft Max after it).

We can add more functions, such as other layers or different activation functions, but this will be sufficient for now.

Creating the Optimiser

The next step is to create the optimiser function and associated loss function. We can simply add this to the code after the model has been created, i.e. where `model = Network()`:

```
optimizer = optim.SGD(model.parameters(), lr=0.001)
```

```
loss_function = nn.NLLLoss()
```

This creates a stochastic gradient descent optimiser and a negative likelihood loss function (NLL). The NLL loss function is well suited to classification problems like the one featured in this example. There are other optimisers we could use, but SDG is an easy one to start with.

Define the Training Loop

We are now ready to create the basic training loop for the DNN. You will see there is already code there to iterate through `epochs` and the training dataset (`x_train, y_train`). At the beginning of the loop we need to zero out the

optimiser gradients to make sure that new gradients are calculated each time around the loop. You can do this with the following code:

```
optimizer.zero_grad()
```

After this you will need to add a call to the `model` after the point the `image` has been created in the existing code, i.e.:

```
output = model(image)
```

Once you have the output you can also calculate the relative loss, using the `loss_function`, the `output`, and the expected value, i.e.:

```
loss = loss_function(output, label)
```

Now we can run the backward propagation/weight update step, using the following code:

```
loss.backward()
```

```
optimizer.step()
```

Finally, record the loss so you can track how the loss is changing over epochs, i.e.:

```
train_loss += loss.item()
```

Congratulations, you have finished your first training loop. You should now be able to run the code and see some changes in the loss amount, give it a try on the login nodes using:

```
python3 pytorch-mnist.py 50
```

Define the Evaluation Loop

The final thing to do for this initial exploration of PyTorch training is create a loop that will run inference (or evaluation) on the test (rather than training) data. The loop is already there, but you now need to adding a forward pass and a calculation as to whether the output from the model matches the label.

Remember, the model will be producing a 10 length tensor, with each entry representing the probability the input was that number. For instance, for the following tensor:

0	1	2	3	4	5	6	7	8	9
0.04	0.12	0.06	0.21	0.09	0.15	0.03	0.10	0.13	0.07

We would record the network has having chosen 3 as the number that was represented by the picture used as input, as it has the highest probability (0.21) compared to the rest of the numbers.

Therefore, you will need to identify the index of the highest value in the output vector and compare that to the provided `label` for the input to see if the network was correct in its predictions.

Try and implement the training loop yourself, and let us know if you have any questions/issues.

Moving to the GPUs

So far we have been running on the EDIF VM CPUs. However, for best performance you will want to use the GPUs. We have provided a yaml file (`run_pytorch_mnist.yaml`) for you to run your job on the GPUs. The file template is below:

```
apiVersion: batch/v1
kind: Job
metadata:
  generateName: mlas-practical2-
  labels:
    kueue.x-k8s.io/queue-name: eidf018ns-user-queue
    eidf/user: "ADDUSERNAMEHERE"
spec:
  completions: 1
  backoffLimit: 0
  ttlSecondsAfterFinished: 1800
  template:
    metadata:
      name: interactive-gpu-access
    spec:
      activeDeadlineSeconds: 600
      containers:
        - name: pytorch
          stdin: true
          image: registry.eidf.ac.uk/eidf018/pytorch:2.10.0-cuda12.8-cudnn9-devel
          imagePullPolicy: Always
          args: ["/bin/bash"]
          resources:
            requests:
              cpu: 2
              memory: '1Gi'
            limits:
```

```

    cpu: 2
    memory: '4Gi'
    nvidia.com/gpu: 1

    volumeMounts:
      - mountPath: /mnt/pvc
        name: volume
    restartPolicy: Never
    imagePullSecrets:
      - name: eidfreq
    volumes:
      - name: volume
        persistentVolumeClaim:
          claimName: ADDPVCNAMEHERE
    nodeSelector:
      nvidia.com/gpu.product: NVIDIA-A100-SXM4-40GB-MIG-
1g.5gb

```

There are two places you will need to adapt this script with your specific information. You should change this line:

```
eidf/user: "ADDUSERNAMEHERE"
```

To replace "ADDUSERNAMEHERE" with your user. For instance, something like this, but with your specific name:

```
eidf/user: "s2935835-mlas"
```

The other place is the following line that specifies your Persistence Volume Claim (PVC), which is the storage you have been given that you can mount on the GPU job you run. If you don't have a PVC you need to request one, email a.jackson@epcc.ed.ac.uk with your username to get one. You PVC will be somewhere such as `/home/eidf018/eidf018/shared/s2935835-mlas-pvc`. You can add it to your yaml file by updating the following line:

```
claimName: ADDPVCNAMEHERE
```

This needs to be updated for your specific PVC name. In general, this will be your user account name with `-pvc` at the end of it, i.e.:

```
claimName: s2935835-mlas-pvc
```

Once you have an updated `yaml` file you can submit it to reserve and use a GPU node. However, first we also need to enable access to the EIDF container registry, which holds the container that we will run

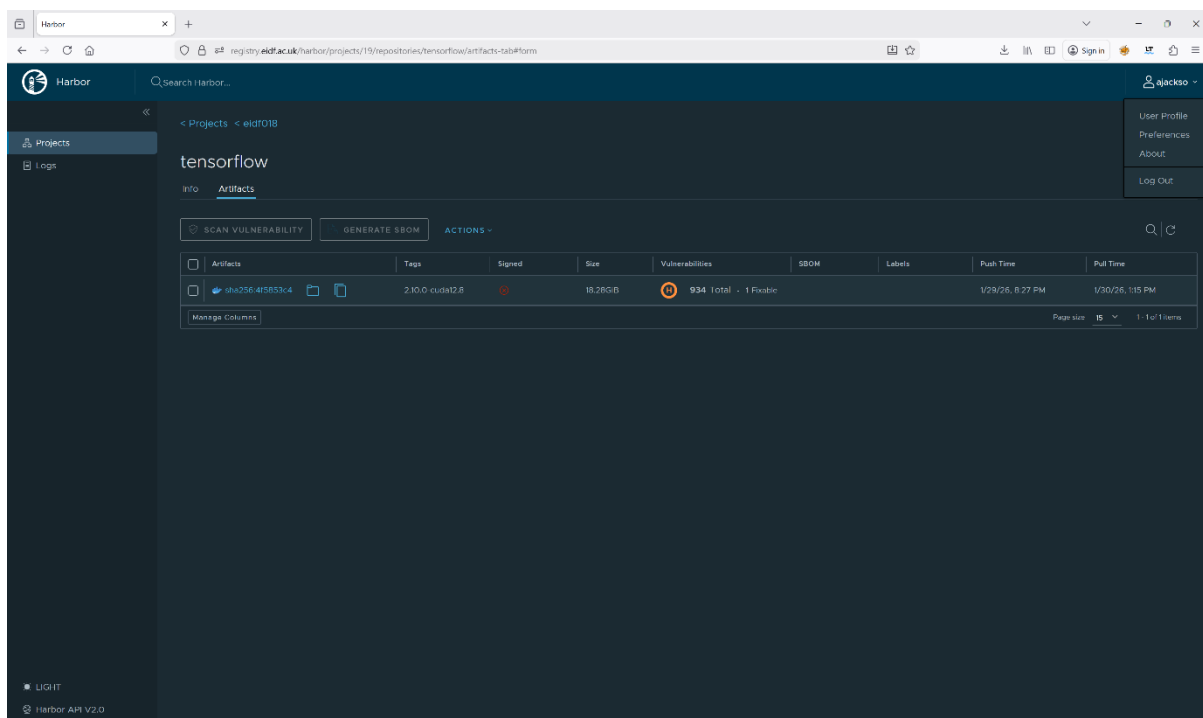
(registry.eidf.ac.uk/eidf018/pytorch:2.10.0-cuda12.8-cudnn9-devel in this instance). To do this you need to go to this website and login (you can use your SAFE username and password to login):

<https://registry.eidf.ac.uk>

Once you have logged in to the website you can go back to your terminal on the EIDF VM for this course and type:

```
docker login registry.eidf.ac.uk
```

This will ask for a username and password. You can get these from the registry website, using the menu in the top right corner, and selecting “User Profile”, as show in this image:



This will bring up a username and a password (secret) as shown in this image:

The image shows a 'User Profile' form with the following fields and values:

Field	Value
Username	ajackso
Email	adrianj@epcc.ed.ac.uk
First and last name	ajackso
Comments	Onboarded via OIDC provider
CLI secret	[Redacted]

At the bottom of the form are two buttons: 'CANCEL' and 'OK'.

Use the “Username” for the docker login username, and the “CLI secret” as the password. Once you have logged in you also have to do some setup to allow kubectl to know you have access to the registry. Follow these instructions:

Firstly, you need to encode your authorisation information into base64 for inclusion in a Kubernetes secret, using the following command on the EIDF VM. This will generate a string of characters:

```
echo '{"auths":{"https://registry.eidf.ac.uk":{"username":"<your username>","password":"<your CLI Secret>"}}}' | base64
```

The CLI Secret value and username are both taken from the registry website as outlined for the docker login command. You will get output that looks something like the following. It may have line breaks on output, and for the next part, you can remove the line breaks and make it a single line:

```
eyJhdXRocyI6eyJodHRwczovL3JlZ2lzdHJ5LmVpZGYuYWVudWsiOnsidXNlcm5hbWUiOiJyb2JvdCR0ZXN0X2xpYnJhcncrZ3B1X2RlZmF1bHQiLCJwYXNzd29yZCI6InZrakRHRzNwSG9lTVc0bEtpTHRTZGt3ZXdlcVlpRDFxc1RPT2cifX19Cg==
```

Now create a YAML file with the following information:

```
apiVersion: v1
kind: Secret
metadata:
  name: eidfreg
  namespace: eidf018ns
  labels:
    eidf/user: "ADDUSERNAMEHERE"
```

```

type: kubernetes.io/dockerconfigjson
data:
  .dockerconfigjson: >-
    eyJhdXRocyI6eyJodHRwczovL3JlZ2lzdHJ5LmVpZGYuYWMudWsiOms
    idXNlcm5hbWUiOiJyb2JvdCR0ZXN0X2xpYnJhcnkrZ3B1X2RlZmF1bH
    QiLCJwYXNzd29yZCI6InZrakRHRzNwSG9lTVc0bEtpTHRTZGt3ZXdlc
    VlpRDFxc1RPT2cifX19Cg==

```

Where you need to replace “ADDUSERNAMEHERE” with your username, and replace everything after >- with the text you generated above running the “echo” command.

To apply the secret to your Kubernetes account (replacing <your filename> with the name of the file you created above):

```
kubectl apply -f <your filename> -n eidf018ns
```

This should conclude the setup of your environment to run on the EIDF. Now we are ready to run a GPU container and access the GPUs. You can do this with the following command:

```
kubectl create -f run_pytorch_mnist.yaml
```

That will report something like that at the command line:

```
job.batch/mlas-practical2-cm66z created
```

You can check that this has started correctly using

```
kubectl get pods
```

This will provide a list of all the jobs (pods) that are running for the eidf018 project, and will look something like this:

NAME	READY	STATUS	RESTARTS	AGE
amv-intermediate-fd2kr-qgqh8	0/1	Completed	0	32h
amv-mid-batchsize-5zn45-z642c	0/1	Completed	0	31h
amv-mid-workers-4n5wf-69p9p	0/1	Completed	0	31h
amv-mid-workers-6m6c2-qhf68	0/1	Completed	0	6h29m
amv-mid-workers-batchsizec84pj-rlbmm	0/1	Completed	0	5h45m
amv-mid-workers-qwpfp-zk78w	0/1	Completed	0	6h15m
bjobtest-s2fnp-7dwxf	1/1	Running	0	3d3h
bjobtest-s2fnp-blngt	0/1	Error	0	8d
bjobtest-s2fnp-fgjvh	0/1	Error	0	13d
green-hpc-test-jwdq6-c4mxj	0/1	Pending	0	11d
juliet-train-dn8nc-dc2w7	0/1	Pending	0	112d
juliet-train-v9k6j-d7swq	0/1	Pending	0	112d

juliet-train-zdrxf-55rdj	0/1	Pending	0	112d
mlas-practical2-cm66z-kwfkv	1/1	Running	0	5m3s
stgcn-test-9nf5z	0/1	Completed	0	8h
stgcn-test-q6cjt	0/1	Completed	0	9h
stgcn-train-full-pipeline-rpd7g	0/1	Completed	0	9h
stgcn-train-full-pipeline-zzj99	0/1	Completed	0	8h

The one that is highlighted above is an example of what you will see for a job you submit. The text matches what you got when submitting the job (i.e. `mlas-practical2-cm66z`) but with an extra `-` and 5 characters added.

You can then access that GPU node using this command:

```
kubectl exec --stdin --tty mlas-practical2-cm66z-kwfkv -- /bin/bash
```

After running this, you should get a command line prompt on the GPU node itself.

For the yaml file you have, we have specified a specific kind of GPU node (NVIDIA-A100-SXM4-40GB-MIG-1g.5gb), for this example that is a single A100 Nvidia GPU, as well as specifying the number of CPUs and amount of memory available to your job (2 cores and up to 4GB of memory). For future practical exercises we might alter these parameters, more details on what is possible are available [here](#):

We have also added some constraints to the job, particularly this line:

```
activeDeadlineSeconds: 600
```

Means that the job will automatically finish after 600 seconds (10 minutes), even if you're still using it. You can resubmit the job after that to get a new job, or if you need to run longer jobs you should edit the 600 seconds above and make it a larger value.

Whilst you can now get a command line prompt on a GPU (using the `exec`) command above, there are some other things you may need to do to be able to run the MNIST code on the system. By default no data or files are in the GPU system. You have mounted your PVC, which gives you a mechanism to access models and data on the GPU.

From the EIDF VM, rather than the GPU node, copy your python MNIST code (`pytorch-mnist.py`) to your PVC (i.e. something like `/home/eidf018/eidf018/shared/s2935835-mlas-pvc`), and copy the data directory (`/home/eidf018/eidf018/shared/mlas/mnist`) there as

well. To make this quicker we've created a tar file of the directory, so you can copy as follows (assuming you are in your PVC directory already):

```
cp /home/eidf018/eidf018/shared/mlas/mnist.tar .  
tar xf mnist.tar
```

Now you should be able to access these on the GPU node. You will need to go to the correct directory, i.e.:

```
cd /mnt/pvc
```

When you're on the GPU node. You will also need to update your code to replace `home/eidf018/eidf018/shared/mlas/mnist` with `/mnt/pvc` in the places where your code loads data.

You should now be able to run your application on the node, i.e.:

```
python3 pytorch-mnist.py 60000
```

However, this will not automatically use the GPUs from PyTorch. You will need to create a device that selects the GPUs and then move the model and data to be used onto the GPU. The lecture notes have details of how to do this, but if you have problems or cannot work out what to do let us know and we will help.

Try running on the CPU and GPU and see what performance difference you experience. Try with a range of different image numbers (modifying the command line parameter to change the number of images used for training and testing).

Extra Exercises

There are a range of extra exercises to undertake. Firstly, try experimenting with different numbers and sizes of layers in the network. Do these impact performance or accuracy?

Try different numbers of epochs to see how training and test accuracy changes with longer training.

Add in some ReLU activation layers in the network, how do they change performance?

Currently, we are using very inefficient data loading and processing approaches. Batching your data into groups and then passing these to the network directly can produce better performance. You could replace the current data loading with the following code to see if it improves performance at all:

```

from torchvision import datasets

...

train =
datasets.MNIST(/home/eidf018/eidf018/shared/mlas/mnist/mnist
_data', download=False, train=True, transform=transform)

test = datasets.MNIST('
(/home/eidf018/eidf018/shared/mlas/mnist/mnist_data',
download=False, train=False, transform=transform)

x_y_train = torch.utils.data.DataLoader(trainset,
batch_size=64, shuffle=True)

x_y_test = torch.utils.data.DataLoader(valset,
batch_size=64, shuffle=True)

```

Note, this would need to be added after the `transform` is defined in your program. If you use this form of data loading you will need to update the main training and inference/evaluation loops, to get the data from the `x_y_train` and `x_y_test` objects instead, i.e. using something like:

```

for images, labels in x_y_train:

```

You can then pass the images and labels data structures to the model and loss functions etc... One of the nice features of PyTorch and similar frameworks is that they automatically can deal with batched data, and this can improve GPU occupancy/performance.

Once you get this working you can try different batch sizes, etc... to see how they impact performance and quality.